

Implementation:

To perform object detection on every 5th frame, I modified the original client's source code's "run" function. Specifically, for every completely reconstructed frame, I checked if the frame count was a multiple of 5. If it was, I sent the image data to the DETR model server and retrieved the response. The DETR model's response included labels, confidence scores, and labels for detected objects. If the model was 0.90 confident or more in its prediction, I overlaid the image with the associated prediction's bounding box, label, and confidence. Finally, I stored the overlaid image in the "src" directory.

Screenshots:

Screenshots from Part1 and Part2 are provided on the next page.

Challenges:

The greatest challenge was using the DETR EMS checkpoint to compile a DETR model engine for my GPU. Initially, I did not know that the engines were GPU specific, so I did not understand why my requests to the DETR server kept failing. When I finally figured out the reason, I had to compile a TensorRT engine my machine could use and update the Docker environment with it. This was challenging given my inexperience with Docker.

Part 1:

```
(base) james-beck@james-beck-Allienware-16-Aurora-AC16250:~/Documents/DSA_Trial_Assignment/cognitive_ens_trial_jaylbeck/egoens-sin$ conda activate egoens_sim
(egoens_sim) james-beck@james-beck-Allienware-16-Aurora-AC16250:~/Documents/DSA_Trial_Assignment/cognitive_ens_trial_jaylbeck/egoens-sin$ python simulator/client/raw_client.py 127.0.0.1 --port 9000 --width 480 --height 270
[client] Connecting to 127.0.0.1:9000...
[client] Connected
[client] First frame OK: video=388800 bytes, audio=1066 bytes, csv=4.838681697845459,-8.343792915344238,1.3646949529647827
[client] frames=11 fps=10.9 bad=0 last_csv=5.025429248809815,-8.485851155090332,1.2042834758758545
[client] frames=41 fps=20.4 bad=0 last_csv=4.845864295959473,-8.633491516113281,1.357512354850769
[client] frames=71 fps=23.6 bad=0 last_csv=5.5976433753967285,-6.026205539703369,0.5434837937355042
[client] frames=102 fps=25.2 bad=0 last_csv=4.5825018882751465,-3.1316158771514893,3.615244626998901
[client] frames=133 fps=26.2 bad=0 last_csv=4.766855716705322,-7.292738437652588,7.4986395835876465
[client] frames=163 fps=26.8 bad=0 last_csv=3.189076662063598,-8.844181060791016,2.516305923461914
[client] frames=193 fps=27.3 bad=0 last_csv=5.631162166595459,-5.566518783569336,5.363011837005615
[client] frames=223 fps=27.6 bad=0 last_csv=6.9024834632873535,-5.882553577423096,4.139574527740479
```

Part 2:

